

XIAMEN UNIVERSITY MALAYSIA
厦門大學馬來西亞分校

二分算法

Binary Search and Binary Search on Answer

作者：Wu Youfan
School of Computing and Data Science, Xiamen University Malaysia

2026 年 10 月 18 日

目录

1. 二分查找基础
2. 二分查找变体
3. 二分答案
4. 总结



目录

1 二分查找基础

2 二分查找变体

3 二分答案

4 总结

什么是二分查找

二分查找：在**有序**数组中，每次将搜索区间缩小一半，直到找到目标元素。

经典生活类比：猜数字

- 数字在 1 到 100 之间，每次猜测后告诉你“大了”还是“小了”。
- 每次都猜中间的数字（50、25、75 ...），最多 7 次就能猜中。
- 这就是二分：每次砍掉一半的错误选项。

前置条件

- 数据必须具有**单调性**（通常要求已排好序）。
- 操作必须能通过一次比较，排除当前一半的区间。

二分的核心：利用单调性，每次排除一半，把 $O(n)$ 变成 $O(\log n)$

例题：查找目标元素

题目：给定一个升序排列的数组 `a[1..n]` 和一个目标值 `k`。请找到数组中等于 `k` 的元素的下标。如果存在多个，返回任意一个即可；如果不存在，输出 -1。

示例 1

- 输入：n = 5, k = 4, 数组为 [1, 3, 4, 6, 7]
- 输出：3（4 在下标 3 的位置）

示例 2

- 输入：n = 4, k = 5, 数组为 [1, 3, 4, 6]
- 输出：-1（找不到 5）

查找目标元素：解法

思路

- 维护区间 `[l, r]`，初始 `l = 1, r = n`。
- 每次取 `mid = (l + r) / 2`。
- 如果 `a[mid] == k`，直接返回下标。
- 如果 `a[mid] > k`，目标在左边，更新 `r = mid - 1`。
- 如果 `a[mid] < k`，目标在右边，更新 `l = mid + 1`。

参考代码

查找目标元素：解法

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

```cpp
int l = 1, r = n, best = -1;
while (l <= r) {
    int mid = (l + r) >> 1;
    if (a[mid] == k) {
        best = mid;
        break;
    } else if (a[mid] > k) {
        l = mid + 1;
    } else {
        r = mid - 1;
    }
}
cout << best << endl;
```

目录

1 二分查找基础

2 二分查找变体

3 二分答案

4 总结

例题：查找第一个大于等于 k 的元素

题目：给定一个升序排列的数组 `a[1..n]` 和一个目标值 `k`。请找到数组中**第一个大于等于** `k` 的元素的下标。

示例 1

- 输入：n = 5, k = 4, 数组为 [1, 3, 5, 7, 9]
- 输出：3（5 在下标 3 的位置，是第一个 ≥ 4 的数）

示例 2

- 输入：n = 5, k = 8, 数组为 [1, 3, 5, 7, 9]
- 输出：5（9 在下标 5 的位置，是第一个 ≥ 8 的数）

示例 3

- 输入：n = 5, k = 10, 数组为 [1, 3, 5, 7, 9]
- 输出：-1（没有大于等于 10 的数）

查找第一个大于等于 k 的元素：解法

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

思路：用 best 记录可能的答案

- 当 `a[mid] >= k` 时，说明 `mid` 可能是一个答案，将其存入 `best`。
- 但我们要找的是“第一个”，所以还要继续向左找，即 `r = mid - 1`。
- 当 `a[mid] < k` 时，说明中值太小，只可能向右找，即 `l = mid + 1`。

参考代码

```
int l = 1, r = n, best = -1;
while (l <= r) {
    int mid = (l + r) >> 1;
    if (a[mid] >= k) {
        best = mid;
        r = mid - 1;   // 继续向左收缩
    } else {
        l = mid + 1;
    }
}
```

查找第一个大于等于 k 的元素：解法


```
11 cout << best << endl;
```

注意：这个模板常被称为“左边界二分”，是 C++ STL 中 `lower_bound` 的底层实现。

目录

1 二分查找基础

2 二分查找变体

3 二分答案

4 总结

什么是二分答案

二分答案：当直接求解答案十分困难，但**验证一个答案是否合法**很容易时，我们可以对“答案的值域”进行二分。

适用特征

- 答案具有单调性。例如：如果高度 H 满足条件，那么比 H 小的 H’ 也一定满足。
- 问题的解存在一个明确的上下界 `[l, r]`。
- 能够写一个 `check(x)` 函数，明确判定 `x` 是否满足条件。

解题步骤

1. 确定答案的单调性（求最大还是求最小）。
2. 设定二分的上下界 `l` 和 `r`。
3. 写出 `check(mid)` 函数，判断当前答案是否合法。
4. 根据 `check(mid)` 的结果，更新 `l` 或 `r`。

二分答案 = 把最优化问题，转化为判定问题

例题：砍树

题目：有 n 棵树，第 i 棵树的高度为 a_i。伐木工需要得到总量至少为 M 的木材。他设置一个伐木高度 H，所有高于 H 的树会被砍掉上面的部分，低于等于 H 的部分保留。求伐木工能达到的最大高度 H。

示例 1

- 输入：n $= 4$, M $= 7$, 树高分别为 20, 15, 10, 17
- 输出：15（当 H=15 时，砍下 $5 + 0 + 0 + 2 = 7$ 木材，刚好满足）

示例 2

- 输入：n $= 5$, M $= 20$, 树高分别为 4, 42, 40, 26, 46
- 输出：36

砍树：解法

四步法

- **状态（答案变量）**：二分的变量是伐木高度 `H`。
- **检查（check）**：当高度为 `h` 时，求所有树砍下的木材总长。如果 $\geq M$，返回 true。
- **边界**：下界 `l = 0`，上界 `r = max(a[i])`。
- **顺序**：求最大高度，所以如果 `check(mid)` 成立，尝试升高高度，即 `l = mid + 1`。

参考代码

砍树：解法

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

```cpp
bool check(int h) {
    long long s = 0;
    for (int i = 1; i <= n; i++) {
        s += max(0, a[i] - h);
    }
    return s >= M;
}

int l = 0, r = mx, best = -1;
while (l <= r) {
    int mid = (l + r) >> 1;
    if (check(mid)) {
        best = mid;
        l = mid + 1;
    } else {
        r = mid - 1;
    }
}
```

目录

1 二分查找基础

2 二分查找变体

3 二分答案

4 总结



总结：二分算法的四要素

1. **单调性**：数据或答案必须存在单调关系，否则无法二分。
2. **上下界**：初始化 `l` 和 `r`，确保答案一定在这个区间内。
3. **mid 计算**：通常用 `(l + r) >> 1`。
4. **条件判断**：依据 `check(mid)` 的结果，决定向哪半边收缩。

常见易错点

- `while(l <= r)` 还是 `while(l < r)`？取决于你如何更新边界。
- 区间为 `[l, r]` 时，更新为 `l = mid + 1` 和 `r = mid - 1`，否则容易死循环。
- 二分答案时，注意数据类型。累加结果可能会超过 `int`，需用 `long long`。

把握单调性，找准边界，写出正确的 check

附录：二分常见陷阱速查

错误	修复
死循环	检查 `l` 和 `r` 的更新是否包含 `mid` 本身
找不到正确答案	确认二分的边界是否正确，或者 `best` 是否被正确更新
check 函数写错	明确题目是求最大值还是最小值，返回条件是否严格
累加溢出	把 `s += a[i]` 中的 `s` 设为 `long long`

边界更新要彻底，check 条件要严密